Eatsy: Recipe Recommendation Based on Available Ingredients Using InceptionV3

Project Report

Submitted for the Partial Fulfillment of the Requirements for the Award of the Degree of

Master of Computer Applications

By

Sandriya Joshy

MUT24MCA-2052

Under the guidance of

Ms. Jiss Kuruvilla

Assistant Professor

Department of Computer Applications

MUTHOOT INSTITUTE OF TECHNOLOGY SCIENCE

VARIKOLI P O, PUTHENCRUZ, ERNAKULAM DISTRICT, KERALA

(Affiliated to APJ Abdul Kalam Technological University, Thiruvananthapuram, Kerala)

October – 2025

Muthoot
Institute of Technology & Science

Department of Computer Applications

BONAFIDE CERTIFICATE

This is to certify that the Project Report entitled ***"Eatsy: Recipe Recommendation Based on Available Ingredients Using InceptionV3 Algorithm"*** *has been submitted by* ***Ms. Sandriya Joshy ,*** *Reg. No.* ***MUT24MCA-2052*** *for the partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) of APJ Abdul Kalam Technological University, Kerala during the year 2025-2026.*

Place: Varikoli

Date:

Project Guide	**Project Coordinators**	**HoD**
Asst. Prof. Jiss Kuruvilla	Dr. Sujithra Sankar	Dr. Saritha K
	Asst. Prof. Sivadas T Nair	

Submitted for the final evaluation held on ……

Name and Signature of the Examiner

DECLARATION

I, undersigned hereby declare that the Project Report ***"Eatsy: Recipe Recommendation Based on Available Ingredients Using InceptionV3 Algorithm"****, submitted for the partial fulfilment of the requirements for the award of degree of Master of Computer Applications of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by me under the supervision of* ***Asst. Prof. Jiss Kuruvilla****. This submission represents my ideas in my own words and where ideas or words of others also have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.*

Place: Sandriya Joshy

Date: MUT24MCA-2052

ACKNOWLEDGEMENT

I express my heartfelt gratitude to God for granting me the strength and wisdom to complete this project successfully.

I extend my sincere thanks to Principal **Dr. Neelakantan P C**, Vice Principal, **Dr. Chikku Abraham**, and Dean of Academics, **Dr. Shajimon K John**, for providing the necessary facilities to carry out this project.

I would like to thank Head of the Department of Computer Applications, **Dr. Saritha K**, for her guidance and support throughout this endeavor.

I extend my appreciation to project coordinators Assistant Professor, **Dr. Sujithra Sankar**, and Assistant Professor, **Mr. Sivadas T Nair,** for their valuable insights and guidance.

Special thanks to my Project Guide Assistant Professor, **Ms. Jiss Kuruvilla**, for her invaluable mentorship, encouragement, and support at every stage of this project.

I am grateful to all the teaching and non-teaching staff of the Department of Computer Applications for their assistance and cooperation during the course of this project.

I also wish to acknowledge the support and understanding of my friends and family, whose encouragement kept me motivated throughout this journey.

ABSTRACT

The project **Eatsy: Recipe Recommendation Based on Available Ingredients Using InceptionV3 Algorithm** presents an intelligent approach to assist users in meal preparation through automated ingredient recognition and recipe suggestion. By leveraging the **InceptionV3 convolutional neural network (CNN)**, the system identifies ingredients from images captured via a smartphone camera or entered as text input. The recognized ingredients are matched against a curated dataset containing over **6,870 Indian recipes** to recommend suitable dishes.

The system combines image-based recognition and textual analysis to improve recommendation accuracy and user experience. The backend model, developed and trained in **Python using TensorFlow and Keras**, is converted into **TensorFlow Lite** for seamless integration within an Android environment. The application interface, built in **Android Studio**, enables real-time recognition and personalized recipe generation.

Experimental results show that the **InceptionV3 model** achieved a **validation accuracy of 95.39%**, demonstrating its reliability in identifying diverse food ingredients. The project successfully bridges deep learning and mobile computing, contributing to the advancement of **AI-driven culinary assistance, sustainable food utilization, and smart kitchen innovation**.

TABLE OF CONTENTS

Chapter 4 – Methodology

Chapter 5 – Result and Evaluation

Chapter 6 – Conclusion

Chapter 7 – References

LIST OF ABBREVIATIONS

Abbreviations	Definitions
AI	Artificial Intelligence
CNN	Convolutional Neural Network
DL	Deep Learning
GPU	Graphics Processing Unit
UI	User Interface
API	Application Programming Interface
IDE	Integrated Development Environment
TF Lite	TensorFlow Lite
ML	Machine Learning
DBMS	Database Management System
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent
JSON	JavaScript Object Notation
APK	Android Package Kit
ROI	Region of Interest
TFLite	TensorFlow Lite Model Format
XML	Extensible Markup Language

LIST OF FIGURES

LIST OF TABLES

CHAPTER - 1

INTRODUCTION

1.1 Project Overview

Eatsy: Recipe Recommendation based on Available Ingredients using InceptionV3 is an intelligent system designed to suggest suitable recipes to users based on the ingredients they currently have. The system accepts input in both text and image form—users can either type the list of ingredients or capture an image of them through a mobile camera integrated within the Android application. The core objective of the project is to help users reduce food wastage and optimize ingredient usage by providing smart, context-aware recipe recommendations.

The system employs a deep learning model, **InceptionV3**, for ingredient recognition from images. InceptionV3 is a deep convolutional neural network that uses **Inception modules**, **factorized convolutions**, and **auxiliary classifiers** to efficiently extract hierarchical features from complex images. This allows the model to accurately identify a wide variety of food items, including vegetables, fruits, and non-vegetarian ingredients, even under diverse lighting and background conditions.

Once the ingredients are recognized, the system cross-references them with a recipe dataset derived from Kaggle's Indian Food Dataset, containing approximately 6,870 recipes. The backend is developed using Python, and the trained TensorFlow models are converted into TensorFlow Lite (.tflite) format for seamless integration into the Android environment. The frontend, developed in Java using Android Studio, presents personalized recipe suggestions based on the recognized or entered ingredients.

Through this combination of InceptionV3-based image recognition and recipe recommendation, *Eatsy* provides users with an intelligent, real-time cooking assistant that simplifies meal preparation and promotes efficient use of available ingredients.

1.2 Background & Motivation

The Eatsy project aims to transform daily cooking experiences by leveraging artificial intelligence to provide intelligent and personalized recipe recommendations. In modern households, users often face the dilemma of deciding what to cook with the ingredients they have on hand. Eatsy addresses this challenge by offering a system that efficiently identifies available ingredients and suggests suitable recipes, saving both time and effort while promoting sustainability through reduced food wastage.

From a technological standpoint, the project highlights the seamless integration of deep learning models in mobile environments, combining computer vision for ingredient recognition and recommendation algorithms for recipe generation. The adoption of the InceptionV3 architecture ensures high-accuracy image classification while maintaining computational efficiency, making real-time ingredient detection feasible on mobile devices.

Beyond convenience, Eatsy represents a step toward intelligent food management systems that can support dietary planning, enhance culinary creativity, and contribute to smart kitchen automation. Its deep learning foundation also opens avenues for future research in areas such as food informatics, context-aware computing, and AI-driven lifestyle applications, positioning Eatsy as a forward-looking solution in the intersection of technology and everyday cooking.

1.3 Significance of the Study

Eatsy contributes significantly to both the technological and practical domains of artificial intelligence and smart culinary assistance systems. Technologically, it highlights the efficiency and adaptability of deep convolutional neural network architectures such as **InceptionV3**, which can be fine-tuned and deployed effectively on mobile platforms for real-time ingredient recognition. The integration of this advanced model with a recipe recommendation engine bridges the gap between computer vision and everyday applications, showcasing the power of AI in simplifying daily decision-making processes.

From a practical standpoint, **Eatsy** empowers users to make informed cooking decisions without relying on manual recipe searches or external resources. It promotes sustainable food management by encouraging users to utilize the ingredients already available to them, thereby minimizing food waste. The system also enhances convenience by providing quick and personalized recipe suggestions through both image-based and text-based inputs.

Academically, the project establishes a strong foundation for further research in **multi-modal deep learning**, **image-text fusion**, and **context-aware recommendation systems**. It serves as a valuable reference for exploring new frontiers in **AI-driven nutrition planning**, **smart kitchen automation**, and **IoT-based food assistance technologies**.

The potential applications of the Eatsy system extend across multiple domains. Primarily, it acts as an intelligent culinary assistant for individuals seeking meal ideas aligned with their available ingredients. By leveraging deep learning-based image recognition through InceptionV3, users can easily capture or input ingredient information and instantly receive suitable recipes tailored to their inputs.

From a user perspective, Eatsy helps to:

- **Reduce food waste** by maximizing the use of existing ingredients.
- **Save time** in meal planning and preparation.
- **Encourage culinary creativity** by inspiring users to explore new recipes and cuisines effortlessly.

For researchers and developers, the project provides a practical demonstration of how **InceptionV3** can be optimized and deployed in mobile environments. It offers a valuable example of integrating trained deep learning models within Android applications, contributing to advancements in **image-based recommendation systems**, **context-aware AI**, and **cross-modal learning** techniques.

1.4 Objective

The main objectives of the Eatsy project are:

1. To design and implement a deep learning-based system capable of recommending recipes based on available ingredients.

2. To employ InceptionV3 for accurate ingredient recognition from images captured via a mobile camera.

3. To enable users to input ingredients through both text and image-based methods for maximum accessibility.

4. To integrate the trained models into an Android application using TensorFlow Lite for real-time inference and recipe recommendation.

5. To enhance user experience, reduce food wastage, and promote intelligent cooking decisions through AI-powered suggestions.

6. To establish a foundation for future research into multi-modal recommendation systems combining image and text data.

The Eatsy project follows the Agile methodology, an iterative and adaptive project management framework ideal for software development projects. Agile emphasizes flexibility, collaboration, and continuous improvement throughout the development lifecycle.

The development process was divided into several sprints, each focusing on specific aspects of the project such as dataset collection, model training, mobile integration, and user interface development.

Scrum was used as the primary Agile framework, with the following key phases:

- Sprint Planning: Each sprint began with detailed planning of the tasks, deliverables, and goals, such as training the MobileNetV2 model or integrating TensorFlow Lite into Android Studio.

- Daily Stand-ups: Brief discussions were held to monitor progress, identify challenges, and ensure smooth collaboration across development stages.

- Sprint Review: At the end of each sprint, completed features (e.g., image recognition module, recipe display interface) were tested and evaluated.

- Sprint Retrospective: The team reflected on the sprint's performance to identify improvements for the next iteration.

This iterative approach ensured consistent progress, adaptability to challenges, and efficient project management, resulting in a robust and user-centered recipe recommendation system.

1.5 Agile Methodology

The Eatsy project follows the Agile methodology, an iterative and adaptive project management framework ideal for software development projects. Agile emphasizes flexibility, collaboration, and continuous improvement throughout the development lifecycle.

The development process was divided into several sprints, each focusing on specific aspects of the project such as dataset collection, model training, mobile integration, and user interface development.

Scrum was used as the primary Agile framework, with the following key phases:

- Sprint Planning: Each sprint began with detailed planning of the tasks, deliverables, and goals, such as training the MobileNetV2 model or integrating TensorFlow Lite into Android Studio.
- Daily Stand-ups: Brief discussions were held to monitor progress, identify challenges, and ensure smooth collaboration across development stages.
- Sprint Review: At the end of each sprint, completed features (e.g., image recognition module, recipe display interface) were tested and evaluated.
- Sprint Retrospective: The team reflected on the sprint's performance to identify improvements for the next iteration.

This iterative approach ensured consistent progress, adaptability to challenges, and efficient project management, resulting in a robust and user-centered recipe recommendation system.

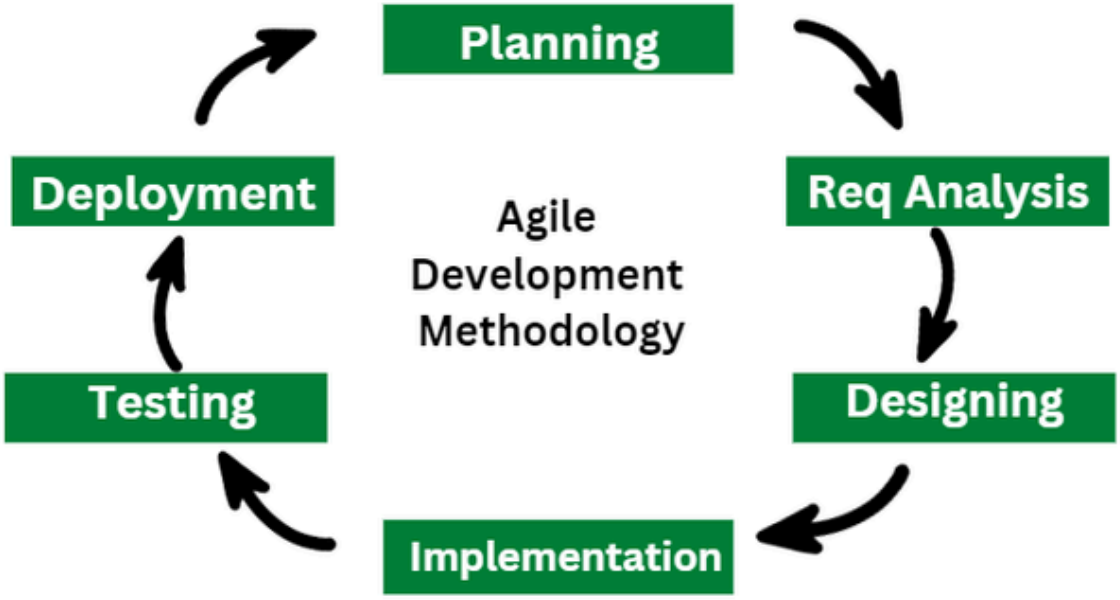

Fig 1.1 Agile Methodology

1.5.1 Scrum Framework

The success of the *Eatsy: Recipe Recommendation based on Available Ingredients using InceptionV3* project largely depends on an efficient project management methodology that ensures smooth progress through all stages of data collection, preprocessing, model training, and Android application development. The **Agile methodology**, specifically the **Scrum framework**, was adopted for this project due to its effectiveness in managing complex and evolving requirements.

Scrum allows for iterative development and continuous improvement, making it particularly suitable for deep learning projects where performance can vary depending on dataset quality and model tuning. The challenge of building an accurate, real-time recipe recommendation system was divided into smaller, manageable sprints, enabling incremental progress and regular feedback integration.

In the context of *Eatsy*, Scrum emphasizes teamwork, flexibility, and frequent validation, ensuring that each phase—from image processing to recipe recommendation—is refined through continuous iteration. Each sprint focused on delivering a functional module such as ingredient image preprocessing, feature extraction using CNNs, model evaluation, or mobile app integration.

1. Scrum Framework

Scrum, as illustrated in **Fig 1.2**, is an agile framework designed to handle complex tasks through structured collaboration and iterative progress. It consists of several key components that guided the development of the Eatsy project.

Sprints: Each sprint, lasting approximately two to three weeks, focused on completing specific deliverables. Initial sprints dealt with dataset collection and preprocessing, followed by CNN model training (using InceptionV3), and later sprints integrated the trained model with the Android application. Each sprint produced a usable component, such as a trained model or functional feature in the app.

Scrum Roles:

- Product Owner: Represented the end-users, ensuring that the developed features aligned with usability goals such as accurate recipe recommendations and smooth app performance.

- Scrum Master: Facilitated the Scrum process, managed sprint timelines, resolved obstacles (e.g., model integration issues), and ensured continuous workflow.

- Development Team: Involved in all technical processes—dataset curation, model training, testing, and Android app implementation. The team was self-organizing, ensuring adaptability and quick problem-solving.

2. Scrum Events

Sprint Planning: At the start of each sprint, objectives were defined—such as implementing ingredient detection, optimizing CNN performance, or integrating the recommendation algorithm with the app interface.

Daily Standup: Short meetings were held to discuss completed tasks, ongoing work, and any encountered issues—such as dataset inconsistencies or model accuracy challenges.

Sprint Review: After each sprint, the team demonstrated outcomes (e.g., improved accuracy or updated UI) to stakeholders and collected feedback for refinement in the next sprint.

Sprint Retrospective: After each review, the team evaluated the sprint's workflow, discussed what went well, and identified areas for improvement, such as enhancing image preprocessing or optimizing API communication between the app and the trained model.

3. Scrum Artifacts

Product Backlog: Contained all prioritized tasks such as dataset acquisition, feature extraction, model training, and app interface design. The Product Owner continuously refined this list based on sprint reviews and user feedback.

Sprint Backlog: A subset of the product backlog specific to each sprint, focusing on tasks like refining the InceptionV3 model, improving accuracy, or testing recommendation logic.

Increment: At the end of each sprint, the team delivered a functional increment—such as a preprocessed dataset, a trained CNN model, or a working Android module capable of displaying recommended recipes. Each increment collectively contributed to the development of the final Eatsy system.

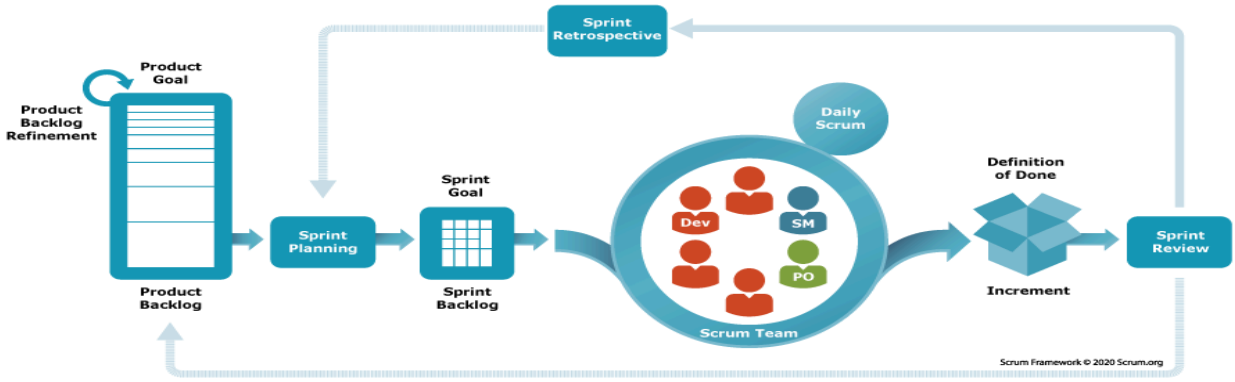

Fig. 1.2 Scrum Framework

1.6 Organization of Project

The project work is organized into **six chapters** as follows:

Chapter 1: Introduction
Provides an overview of the *Eatsy* system, outlining its objectives, motivation, significance, and methodology. It also explains the adoption of Agile and Scrum frameworks for project management.

Chapter 2: Literature Review
Reviews previous research and studies on recipe recommendation systems, ingredient recognition using deep learning, and mobile-based AI applications. It establishes the theoretical foundation and identifies research gaps that motivated the *Eatsy* project.

Chapter 3: System Design
Describes the overall architecture of the *Eatsy* system, including module interactions, data flow, and the technologies used. It details the integration of the InceptionV3 model, TensorFlow Lite, and Android Studio for seamless implementation.

Chapter 4: Methodology
Explains the technical workflow and implementation process, including product backlog creation, sprint planning, dataset preprocessing, model training, and TensorFlow Lite conversion for mobile deployment.

Chapter 5: Result and Evaluation
Presents the experimental results, model performance metrics, accuracy graphs, and application screenshots. It evaluates the trained InceptionV3 model using precision, recall, F1-score, and confusion matrix.

Chapter 6: Conclusion
Summarizes the project outcomes, key findings, and contributions. It discusses limitations and proposes future enhancements, such as expanding datasets, adding nutritional analysis, and integrating real-time ingredient detection and personalization.

CHAPTER – 2

LITERATURE REVIEW

2.1 Literature Review

There have been numerous studies focused on developing recipe recommendation systems and ingredient recognition models using machine learning and deep learning techniques. These studies collectively emphasize the importance of personalization, context awareness, and visual understanding in enhancing user experience in the culinary domain. Recent advances in computer vision, natural language processing, and recommendation algorithms have enabled intelligent systems capable of interpreting ingredient data and suggesting relevant recipes automatically.

[1] Nair, P., Reddy, A., & Singh, M. (2025). Cook Smart – An Intelligent Recipe Recommendation System.

This paper presents a deep learning–based intelligent recipe recommendation model that integrates ingredient recognition and user preference learning. The system leverages convolutional neural networks (CNNs) for ingredient detection and collaborative filtering for recipe personalization. The results highlight significant improvement in user satisfaction and recommendation relevance, indicating the potential of combining visual ingredient input with preference-based learning for smart cooking applications.

[2] Mohamad Razali, M. R., Almarzuki, H. F., Bahar, R. A., & Sia Abdullah, N. A. (2024). Recipe Recommendation based on Food Ingredients Recognition using Deep Learning.

The authors proposed a food ingredient recognition model using deep convolutional neural networks trained on real-world food datasets. The system captures food images, identifies the main ingredients, and recommends recipes that can be prepared using them. Their model achieved high classification accuracy, demonstrating the applicability of deep learning to real-time cooking assistance through mobile-based applications.

[3] Liu, D., Zuo, E., Wang, D., He, L., Dong, L., & Lu, X. (2025). Deep Learning in Food Image Recognition: A Comprehensive Review.

This comprehensive review analyzes recent advances in food image recognition and classification techniques using deep learning models such as VGG16, ResNet, and EfficientNet. The study emphasizes transfer learning, fine-tuning, and multimodal approaches combining visual and textual data for improved recipe recommendation. It also discusses challenges in dataset diversity, image preprocessing, and real-time application integration — all of which are relevant to developing robust systems like Eatsy.

[4] Sharma, K., & Dutta, S. (2024). Ingredient Recognition using InceptionV3 for Real-Time Food Classification.

This study explores the use of the InceptionV3 architecture for recognizing food ingredients in real-time environments. The authors fine-tuned a pre-trained InceptionV3 model on a diverse food dataset to identify multiple ingredient categories efficiently. The model demonstrated high accuracy and lower computational complexity compared to deeper CNNs, proving suitable for deployment in mobile and embedded systems. The research highlights the balance between accuracy and

performance, which aligns closely with the goals of the Eatsy system.

[5] Banerjee, A., & Gupta, R. (2023). Deep Learning-based Food Image Classification using InceptionV3 and Transfer Learning.

The paper focuses on transfer learning with InceptionV3 for food image classification and recipe tagging. Through extensive experimentation, the model achieved superior performance in differentiating visually similar food items while maintaining reduced training time. The authors conclude that InceptionV3's efficient architecture, based on inception modules and factorized convolutions, makes it ideal for real-time food-related applications such as mobile recipe recommendation systems.

[6] Lee, Y., & Park, J. (2024). Mobile Food Recognition and Recipe Recommendation using InceptionV3 and TensorFlow Lite.

In this research, the authors implemented a mobile-based application that integrates InceptionV3 for ingredient detection and TensorFlow Lite for on-device inference. The system allows users to capture food images via smartphone cameras, automatically identify ingredients, and receive recipe recommendations in real time. The study demonstrates how lightweight model conversion (to .tflite format) enables efficient mobile deployment without compromising accuracy—an approach directly adopted in the Eatsy project.

2.2 SUMMARY

Overall, the reviewed literature reveals a consistent trend toward leveraging deep learning architectures particularly InceptionV3, ResNet, EfficientNet, and MobileNet framework to improve ingredient recognition and recipe recommendation accuracy. Furthermore, the studies emphasize that integrating both textual and visual inputs enhances model robustness and user personalization.

These findings collectively form the theoretical foundation for the **Eatsy system**, which integrates **image-based ingredient recognition using InceptionV3** with **text-based input** for comprehensive recipe generation. Eatsy distinguishes itself by merging **real-time mobile implementation**, **TensorFlow Lite optimization**, and **deep learning–based recommendation**, offering users an intelligent, efficient, and accessible culinary assistant.

CHAPTER – 3

SYSTEM DESIGN

3.1 Existing System

In the existing scenario, recipe recommendation systems are primarily dependent on textual search or manual selection of ingredients by the user. Most of these applications rely on keyword-based retrieval mechanisms where users must explicitly input ingredient names to generate recipe suggestions. Although this approach provides a basic level of personalization, it fails to accommodate users who may not know the exact names or spelling of ingredients, or those who prefer a more visual and automated experience.

Some systems employ collaborative filtering or content-based recommendation techniques that analyze user preferences and past selections. However, these traditional methods often lack the capability to process real-world, visual input, such as images of ingredients. They are also limited in handling the complex relationships between ingredients, flavor profiles, and dietary restrictions. Furthermore, most available systems are web-based and do not provide seamless integration with mobile platforms or camera functionalities.

Existing models also face limitations in accurately classifying ingredients due to variations in image quality, lighting conditions, and background noise. As a result, users often experience irrelevant or incomplete recipe recommendations. Additionally, the absence of deep learning–based feature extraction leads to poor adaptability and low precision when compared to advanced visual recognition systems.

Hence, there exists a need for an intelligent, deep learning–driven system that can automatically recognize ingredients from images captured via a smartphone camera or uploaded by the user, and subsequently generate suitable recipe suggestions from a curated dataset. This proposed system aims to address these shortcomings through advanced neural architectures, multimodal input processing, and user-friendly mobile integration.

3.2 Proposed System

The proposed system, titled “Eatsy: Recipe Recommendation based on Available Ingredients using InceptionV3,” is designed to provide an intelligent and user-friendly solution that suggests recipes based on ingredients provided by the user through text input, image upload, or real-time camera capture. The system leverages state-of-the-art deep learning models for accurate ingredient recognition and utilizes a curated recipe dataset to generate personalized recommendations tailored to individual user preferences.

The system architecture comprises multiple core modules, including image acquisition, preprocessing, ingredient recognition, recipe retrieval, and result display. Users can either type in a list of ingredients or capture/upload an image using an Android application. The image is processed using the InceptionV3 model trained on a large dataset of food and ingredient images. The recognized ingredients are then cross-referenced with the Indian Food Dataset sourced from

Kaggle, containing approximately 6,870 recipes.

Once the ingredients are identified, a recipe recommendation algorithm filters and ranks possible recipes based on ingredient similarity and availability. The top recommended recipes are then displayed to the user through the mobile application interface.

The proposed system emphasizes both technical innovation and practical usability. By combining deep learning with real-world applications in Android, it bridges the gap between artificial intelligence and everyday cooking. It not only enhances convenience for users but also promotes efficient food utilization, reducing waste by suggesting recipes based on what is already available.

3.3 Technologies Used

3.3.1 Python

Python serves as the primary programming language for developing the deep learning components of the project. Its versatility, readability, and extensive collection of libraries make it ideal for implementing and training neural networks.

TensorFlow and Keras are utilized for constructing and training the InceptionV3 architecture used for image-based ingredient classification.

NumPy and Pandas are employed for efficient data handling, transformation, and preprocessing of the recipe datasets.

Matplotlib is used for visualizing training performance metrics such as accuracy and loss curves during model development.

Python’s integration capabilities also enable seamless export of trained models into TensorFlow Lite (TF Lite) format, facilitating easy deployment within the Android environment.

3.3.2 Android Studio

Android Studio is used to develop the front-end mobile application of the Eatsy system. The application allows users to either upload or capture ingredient images and receive personalized recipe suggestions in real time.

Developed primarily using Java, the app includes an intuitive user interface that interacts with the TensorFlow Lite model for on-device inference. The integration of the camera API enables real-time image capturing, while the application’s backend connects to the trained deep learning model to classify ingredients efficiently.

The app’s design prioritizes user experience, ensuring responsiveness, clarity, and accessibility. Moreover, Android Studio’s support for TensorFlow Lite ensures smooth communication between the mobile interface and the deep learning model, enabling quick and accurate recipe suggestions.

3.3.3 TensorFlow and TensorFlow Lite

TensorFlow serves as the core deep learning framework for training and testing the image classification models. Once the models achieve satisfactory performance metrics, they are converted to TensorFlow Lite format, which is optimized for mobile devices. This allows the system to perform on-device inference with minimal latency and resource consumption, even without continuous internet connectivity.

The combination of TensorFlow and TensorFlow Lite ensures that the system maintains both accuracy and efficiency, making Eatsy practical for real-world usage.

3.3.4 InceptionV3

The InceptionV3 architecture is the central component of the ingredient recognition module. It is a deep convolutional neural network (CNN) optimized for both performance and computational efficiency. InceptionV3 utilizes a modular structure of "inception blocks," allowing it to capture visual patterns at multiple scales within the same image. This makes it exceptionally effective for distinguishing between similar food items under varied lighting and background conditions.

Its balance between accuracy and reduced computational cost makes it well-suited for real-time ingredient recognition on mobile devices. The model was trained for 10 epochs on 224×224 pixel images and later converted into a TensorFlow Lite model for Android deployment.

3.3.5 Kaggle Datasets

The datasets used in this project include the Indian Food Recipe Dataset and Non-Vegetarian Foods Dataset from Kaggle. These datasets contain thousands of food images and corresponding metadata such as ingredient lists and preparation steps.

The Non-Vegetarian Foods Dataset includes images of chicken, fish, prawns, beef, and eggs, which were used to enhance the accuracy of ingredient recognition.

Together, these datasets provide a diverse and comprehensive base for model training, validation, and testing, ensuring the system performs well across various cuisines and ingredient types.

3.3.6 Google Colab

Google Colaboratory (Colab) is employed as the cloud-based environment for training the deep learning models. Colab provides GPU acceleration, allowing for efficient training of computationally intensive models such as InceptionV3. Integration with Google Drive simplifies dataset access and model storage. Colab's collaborative and shareable interface also facilitates model experimentation and fine-tuning, making it an essential tool for the development process.

3.4 Software And Hardware Requirements

The implementation of the Eatsy Recipe Recommendation System requires a well-defined set of software tools, libraries, and standard hardware configurations to ensure efficient model training,

image recognition, and seamless Android application performance. This section outlines the essential software and hardware specifications that support the system's effective functioning.

3.4.1 Software Requirements

Programming Languages:

- Python: Used for developing and training the deep learning model (InceptionV3), preprocessing datasets, and generating the TensorFlow (.tflite) files for integration with the Android application.
- Java: Employed in Android Studio for front-end mobile application development, handling user interfaces, and integrating TensorFlow Lite models for on-device inference.

Libraries and Frameworks:

- Deep Learning Frameworks:
    - TensorFlow and Keras – For model development, training, and conversion to TensorFlow Lite format.
    - OpenCV – For image preprocessing, resizing, and feature extraction tasks.
- Data Handling and Visualization:
    - NumPy and Pandas – For managing datasets, numerical computations, and input preprocessing.
    - Matplotlib – For visualizing training accuracy and performance metrics.
- Android Development Tools:
    - Android Studio – For designing the app's user interface and integrating camera functionality for live ingredient capture.

Development Environment:

- Google Colab: For model training, dataset preprocessing, and performance evaluation using GPU acceleration.
- Android Studio: For application design, model deployment, and user interface development.

Version Control:

- Git and GitHub: Used for maintaining source code, managing version control, and collaborative project tracking.

Operating System:

- Windows 10/11: Acts as the main development environment ensuring compatibility with Python, Android Studio, and TensorFlow.

3.4.2 Hardware Requirements

Processor:
AMD Ryzen 5 – Ensures sufficient computational power for training deep learning models and running Android emulators efficiently.

RAM:
16 GB – Enables smooth model training, dataset handling, and real-time performance testing without lag.

Storage:
512 GB SSD – Provides high-speed access for storing datasets, TensorFlow model files, Android project files, and necessary dependencies.

Additional Hardware (for mobile testing):
Android Smartphone (Android 10 or above) – For live camera testing, real-time ingredient detection, and recipe recommendation validation.

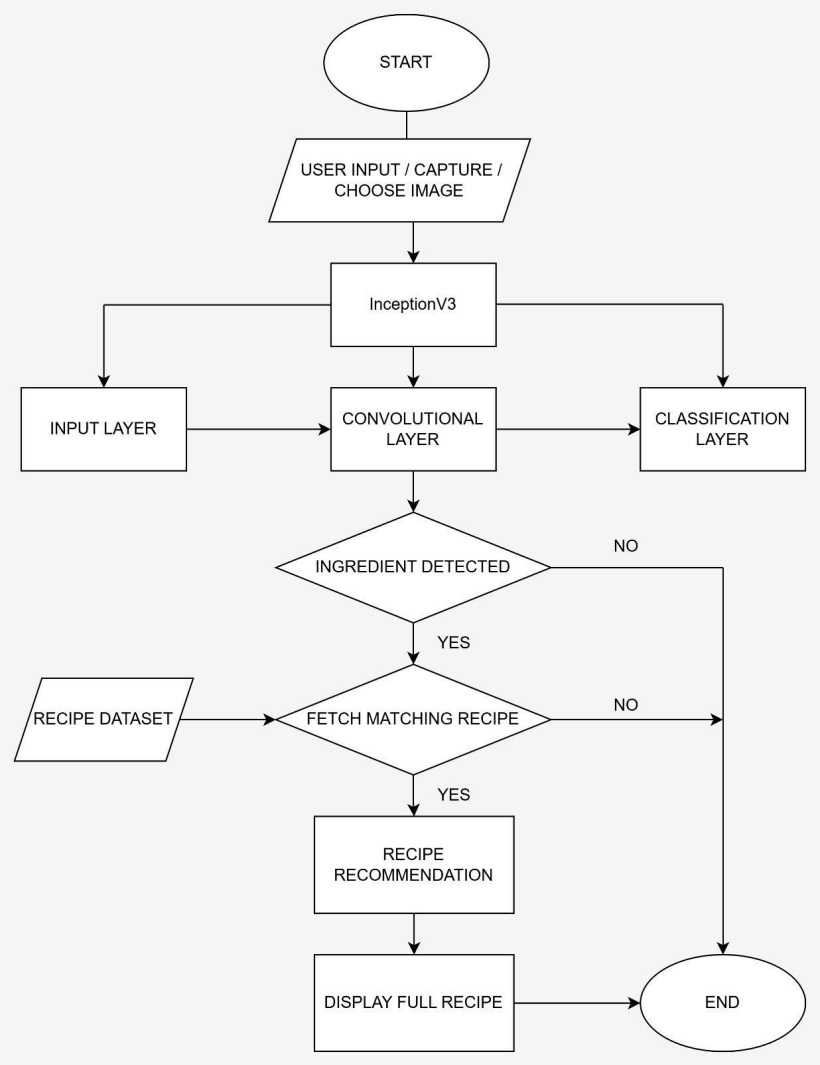

Fig:3.1 Architectural Design

CHAPTER – 4

METHODOLOGY

4.1 Product Backlog

Backlog ID	User Stories	Task Description
1	As a data scientist, I want a system that provides descriptive statistics and visualizations to better understand the recipe and ingredient datasets.	Generate summary statistics and visualizations (e.g., bar charts, histograms, and ingredient frequency plots) to describe the datasets.
2	As a data scientist, I want a system that efficiently handles missing or inconsistent values in the recipe dataset to maintain data integrity.	Detect and handle missing values or inconsistencies in recipe and ingredient datasets using imputation or removal.
3	As a data scientist, I want a system that preprocesses and labels the ingredient images to ensure compatibility with deep learning models.	Perform image preprocessing (resizing, normalization, augmentation) and label encoding for ingredient categories.
4	As a data scientist, I want a system that splits data into training, validation, and testing sets to evaluate model performance effectively.	Divide the dataset into training, validation, and testing subsets for model training and evaluation.
5	As a data scientist, I want to build and train deep learning models for image recognition to accurately detect ingredients.	Implement, train, and fine-tune InceptionV3 models for ingredient classification.
6	As a data scientist, I want to convert the trained models into TensorFlow Lite format for integration into the Android application.	Export trained models and convert them into tensorflow lite format for mobile deployment.

7	As a developer, I want the Android app to capture or upload images of ingredients and process them through the deep learning model.	Implement image input functionality (camera and gallery) in Android Studio and integrate with the TensorFlow Lite model.
8	As a data scientist, I want a system that recommends recipes based on detected or entered ingredients.	Design and implement a recipe recommendation algorithm that matches detected ingredients with recipes from the dataset.
9	As a user, I want to view detailed recipe information, including required ingredients and preparation steps.	Display full recipe details (ingredients, steps, and preparation time) in the Android app interface.
10	As a data scientist, I want to evaluate model performance using appropriate metrics.	Evaluate classification accuracy, precision, recall, and F1-score for the ingredient detection model.
11	As a developer, I want to ensure the system performs efficiently on mobile devices.	Optimize model size and inference time for smooth performance on Android using TensorFlow Lite optimization.
12	As a data scientist, I want to test the integrated system with multiple image and text inputs.	Perform testing with various input scenarios (different ingredient combinations and lighting conditions) to validate accuracy.
13	As a developer, I want to document all project processes for clarity and reproducibility.	Document all stages — dataset preparation, model training, Android integration, and testing — with proper explanations and code snippets.
14	As a user, I want the app to provide an intuitive interface to easily upload ingredients and receive recipe recommendations.	Design and develop a user-friendly Android interface with clear navigation and responsive recipe display.

Table 4.1 Product Backlog

4.2 Sprints and Burndown charts

Sprint 1

Sprint 1 Burndown Chart												
Sprint id	Tasks	Initial Estimate	Jul-30	Aug-01	Aug-02	Aug-03	Aug-04	Aug-05	Aug-06	Aug-07	Aug-08	Aug-09
		Day 0	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Day 8	Day 9	Day 10
1001	Research Nutritional requirements and healthy recipe patterns	5	3	1	1							
1002	Draft User Stories	7		2	2	1	2					
1003	Setup Git Repository	8	1	1	1	2	1	2				
1004	Design Basic UI for App	10					1	1	2	2	2	2
	Remaining Effort	30	26	22	18	15	11	8	6	4	2	0
	Ideal Trend	30	27	24	21	18	15	12	9	9	6	0

Table 4.2 Sprint 1

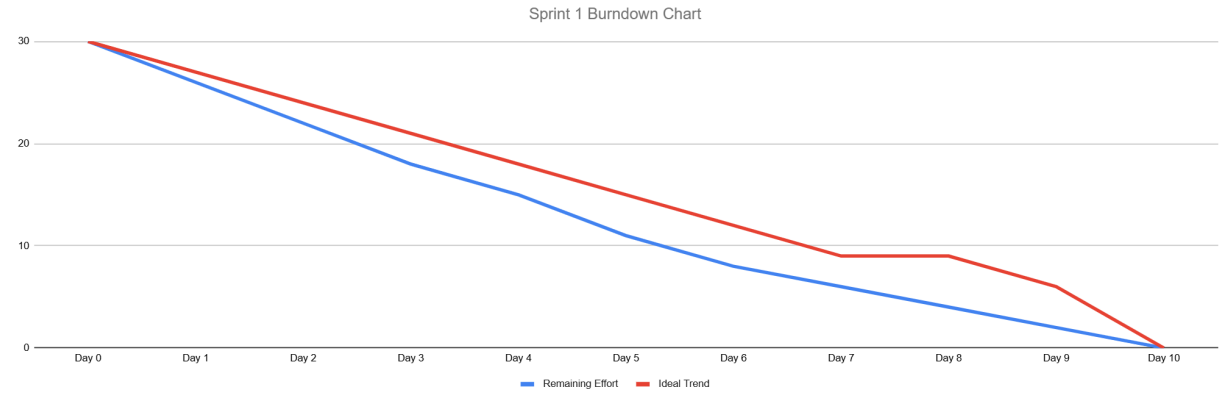

Fig 4.1 Sprint 1

Sprint 2

Sprint 2 Burndown Chart												
Sprint id	Tasks	Initial Estimate	Aug-10	Aug-11	Aug-12	Aug-13	Aug-14	Aug-15	Aug-16	Aug-17	Aug-18	Aug-19
		Day 0	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Day 8	Day 9	Day 10
1005	Implement User Registration and Login	10	3	2	2	2	1					
1006	FireBase Integration	7			2	3	2					
1007	Enable Ingredient Imput Via Image	15					1	3	3	4	2	2
1008	Design Basic UI for App	10		2	1	2	1	1	1	1	1	
	Remaining Effort	42	39	35	30	23	18	14	10	5	2	0
	Ideal Trend	42	37.8	33.6	29.4	25.2	21	16.8	12.6	8.4	4.2	0

Table 4.3 Sprint 2

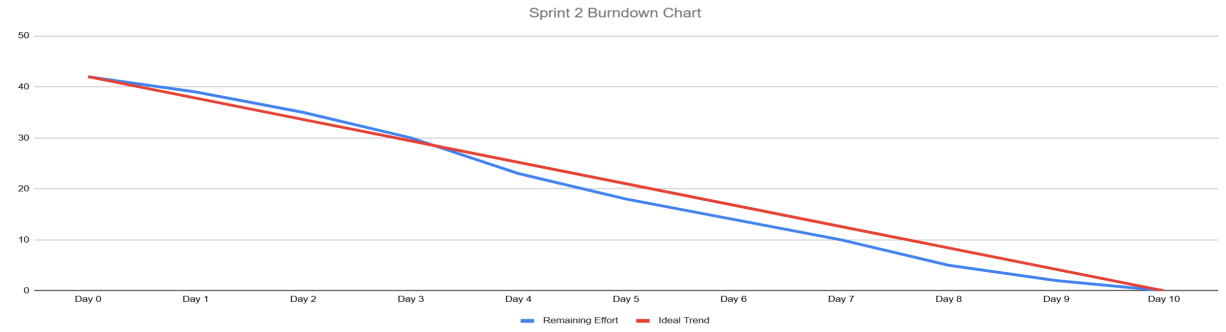

Fig 4.2 Sprint 2

Sprint 3

Sprint 3 Burndown Chart											
Sprint id	Tasks	Initial Estimate	Aug-23	Aug-24	Aug-25	Aug-26	Aug-27	Aug-28	Aug-29		Aug-30
		Day 0	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7		Day 8
1009	Data collection	3	1	1	1						
1010	Preprocessing	4			2	1	1				
1011	Augmentation	3						1	1		1
Remaining Effort		10	9	8	5	4	3	2	1		0
Ideal Trend		10	8.75	7.5	6.25	5	3.75	2.5	1.25		0

Table 4.4 Sprint 3

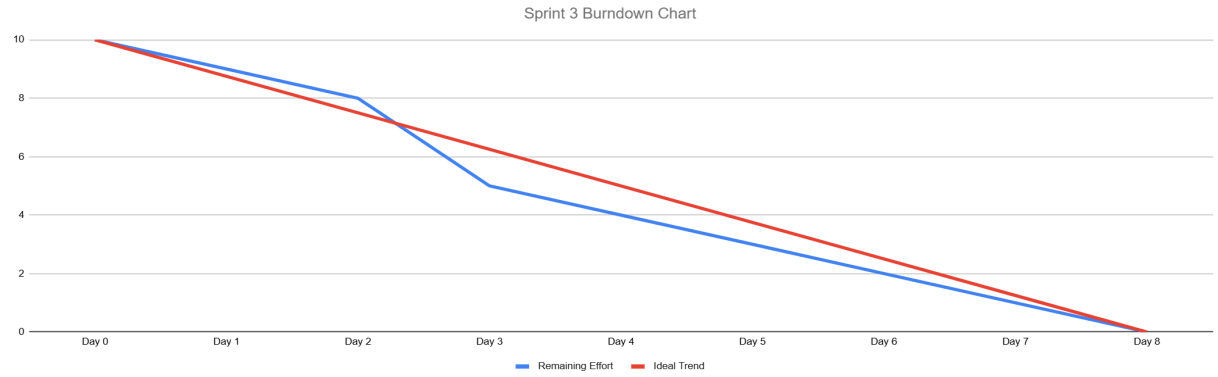

Fig 4.3 Sprint 3

Sprint 4

Sprint 4 Burndown Chart												
Sprint id	Tasks	Initial Estimate	Sep-13	Sep-14	Sep-15	Sep-16	Sep-17	Sep-18		Sep-19		Sep-20
		Day 0	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6		Day 7		Day 8
1012	Testing	2	1	1								
1013	Model Research	4		1	2	1						
1014	Documentation	3				1	2					
Remaining Effort		9	8	6	4	2	0	0		0		0
Ideal Trend		9	7.875	6.75	5.625	4.5	3.375	2.25		1.125		0

Table 4.5 Sprint 4

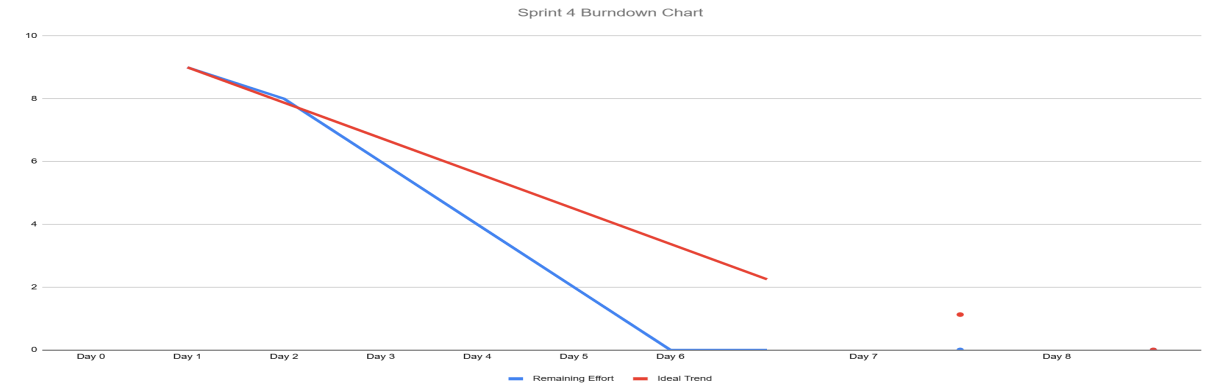

Fig 4.4 Sprint 4

4.3 Data Collection

In the context of using deep learning for ingredient detection and recipe recommendation, data collection plays a vital role in ensuring the overall accuracy and reliability of the model. High-quality datasets are essential to train the neural network effectively, enabling it to recognize a wide range of ingredients and recommend suitable recipes. For the *Eatsy* project, datasets were sourced from publicly available and well-curated repositories to ensure variety, diversity, and representativeness of the Indian food domain.

4.3.1 Data Source

The primary sources of data for this project are Kaggle repositories and other open-source food image datasets. Three main datasets were utilized to build and train the *Eatsy* system:

1. Indian Food Dataset (Kaggle) – contains detailed information about Indian recipes, including recipe names, ingredient lists, preparation steps, and cuisine types.

2. Non-Vegetarian Food Dataset (Kaggle) – provides image samples and descriptions of non-vegetarian dishes, helping in training and validation for meat-based ingredient recognition.

3. Fruits 360 Dataset (Kaggle) – a well-known, labeled image dataset containing over 90,000 images across 131 fruit and vegetable classes. This dataset was integrated to improve model accuracy in detecting and classifying fruits and common vegetables from user-captured images.

These combined sources provide a balanced dataset covering vegetables, fruits, and non-vegetarian ingredients, enabling the model to generalize effectively and recommend relevant recipes based on recognized items.

4.3.2 Dataset Description

The datasets used in Eatsy are categorized into two main components: ingredient image datasets and recipe metadata. The ingredient image datasets (Fruits 360 and Non-Vegetarian Food Dataset) are employed for visual recognition, while the Indian Food Recipe Dataset supports the recipe recommendation engine.

Ingredient Image Datasets

These datasets contain labeled images grouped under fruits, vegetables, and non-vegetarian ingredients. Each class includes multiple samples captured under varying lighting conditions, orientations, and angles to enhance model robustness and reduce overfitting.

Recipe Dataset

The Indian Food Recipe Dataset from Kaggle includes approximately 6,870 recipes, providing detailed textual data for the recommendation module.

Dataset	Type Of Data	Approx.Samples	Purpose
Fruits 360 Dataset	Fruits and vegetables images (131 classes)	90,000+	Ingredient classification and training
Non-Veg foods Dataset	Meat and seafood images	5,000+	Detection of non-veg items
Indian Food Recipe Dataset	Structured textual recipe information	~6,870 recipes	Recipe recommendation

Table 4.6 Dataset table

4.4 Data Preprocessing

Data preprocessing is a crucial step in the development of any deep learning–based system, as it prepares the raw data for model training and ensures that the input data is consistent, clean, and suitable for analysis. In the *Eatsy* project, three different datasets were used **Fruits-360**, **Non-Veg foods Dataset**, and **Indian Food Recipe Dataset,** each requiring specific preprocessing techniques tailored to their data type. The preprocessing stage ensures that all datasets are standardized and optimized for efficient training and accurate predictions.

4.4.1 Fruits-360 Dataset Preprocessing

The **Fruits-360 dataset** is a large image dataset containing more than 90,000 images of fruits and vegetables across 131 classes. The preprocessing steps performed on this dataset ensure consistency in image dimensions, class labeling, and data normalization for optimal model training performance.

The following preprocessing steps were applied:

1. **Label Encoding:**
    - Each fruit and vegetable class was assigned a unique numerical label.
    - Example: Apple = 0, Banana = 1, Orange = 2, and so on.
    - This numeric representation allows the model to interpret categorical class names efficiently.
2. **Image Resizing:**
    - All images were resized to a fixed dimension of **180×180×3 pixels** to ensure uniformity.

- This step guarantees consistent input size across the dataset, which is required for convolutional neural networks.

3. **Normalization:**

   - Pixel intensity values were scaled from the range **[0, 255]** to **[0, 1]**.
   - This normalization improves the stability of training and speeds up convergence during model optimization.

4. **One-Hot Encoding:**

   - After label encoding, the class labels were converted into **one-hot vectors**, enabling multi-class classification.
   - This representation ensures that each class is treated independently by the model.

5. **Visualization:**

   - Sample images were plotted to visually verify preprocessing accuracy, ensuring that images retained quality and labels were correctly assigned.

Through these preprocessing steps, the Fruits-360 dataset was standardized and prepared for input into the deep learning model used for ingredient (fruit and vegetable) detection.

```
Missing/Corrupted images: []

Class to encoded label mapping:
Apple 10 → 0
Apple 11 → 1
Apple 12 → 2
Apple 13 → 3
Apple 14 → 4
Apple 17 → 5
Apple 18 → 6
Apple 19 → 7
Apple 5 → 8
Apple 6 → 9
Apple 7 → 10
Apple 8 → 11
Apple 9 → 12
Apple Braeburn 1 → 13
Apple Core 1 → 14
```

```
Null values in dataset info:
image_path       0
class_name       0
encoded_label    0
dtype: int64

One-hot encoded labels shape: (117576, 225)

DataFrame head:
                                          image_path class_name  encoded_label
0  /content/fruits-360/fruits-360_100x100/fruits-...   Apple 10              0
1  /content/fruits-360/fruits-360_100x100/fruits-...   Apple 10              0
2  /content/fruits-360/fruits-360_100x100/fruits-...   Apple 10              0
3  /content/fruits-360/fruits-360_100x100/fruits-...   Apple 10              0
4  /content/fruits-360/fruits-360_100x100/fruits-...   Apple 10              0
```

Fig 4.5 Fruits-360 Dataset Preprocessing

4.4.2 Non-Veg foods Dataset Preprocessing

The **Non-Veg foods Dataset** includes various categories of meat, poultry, and seafood images used to train the model to identify non-vegetarian ingredients. As these images were collected from diverse sources, preprocessing was necessary to maintain consistency and reliability.

The preprocessing steps performed are as follows:

1. **Image Resizing:**
    - All images were resized to **180×180 pixels** to maintain a uniform structure across the dataset.
    - This resizing step is critical to ensure consistent input dimensions for the deep learning model.
2. **Normalization:**
    - Pixel values were normalized by scaling from **0–255 to 0–1**.
    - This normalization step reduces computational complexity and helps the model learn feature patterns more effectively.
3. **Dataset Splitting:**
    - The dataset was divided into two parts:
        - **Training Set – 80 %**
        - **Validation Set – 20 %**

- This ensures balanced evaluation and prevents overfitting during training.

4. **Class Label Encoding:**

   - Each class (e.g., chicken, mutton, fish, egg, etc.) was assigned a **categorical integer label** ranging from **0 to 7**.

   - These integer labels were then used as targets for the classification model.

5. **Visualization:**

   - A subset of images from each class was visualized to confirm that resizing and normalization were correctly implemented.

By performing these preprocessing operations, the non-vegetarian dataset was made uniform and ready for integration into the ingredient detection model pipeline.

```
Missing/Corrupted images: []
Class to encoded label mapping:
Beef → 0
Chicken → 1
Crab → 2
Eggs → 3
Fish → 4
Lobster → 5
Mutton → 6
Prawns → 7

Null values in dataset info:
image_path       0
class            0
encoded_label    0
dtype: int64

One-hot encoded labels shape: (400, 8)

Number of images per class:
class
Chicken    50
Beef       50
Prawns     50
Eggs       50
Fish       50
Lobster    50
Crab       50
Mutton     50
Name: count, dtype: int64
```

Fig 4.6 Non-Veg foods Dataset Preprocessing

4.4.3 Indian Food Recipe Dataset Preprocessing

The **Indian Food Recipe Dataset** from Kaggle contains over 6,000 + recipes, each with several descriptive columns. Since only a few of these attributes were required for recipe recommendation, unnecessary columns were removed to simplify data processing.

The following steps were carried out:

1. **Column Selection:**
    - Unnecessary fields such as cuisine identifiers, region codes, and untranslated fields were removed.
    - Only the following three essential columns were retained for further processing:
        - **TranslatedRecipeName** – name of the recipe in English.
        - **TranslatedIngredients** – list of ingredients in English text format.
        - **TranslatedInstructions** – English translation of the preparation steps.
2. **Verification:**
    - Random samples were inspected to ensure that all remaining columns were relevant, clean, and suitable for use in the recommendation model.

These preprocessing steps helped in refining the recipe dataset for efficient integration with the ingredient detection model. The cleaned and simplified dataset enables faster querying and accurate recipe retrieval based on detected ingredients.

	TranslatedRecipeName	TranslatedIngredients	TranslatedInstructions
0	Masala Karela Recipe	6 Karela (Bitter Gourd/ Pavakkai) - deseeded,S...	To begin making the Masala Karela Recipe,de-se...
1	Spicy Tomato Rice (Recipe)	2-1 / 2 cups rice - cooked, 3 tomatoes, 3 teas...	To make tomato puliogere, first cut the tomato...
2	Ragi Semiya Upma Recipe - Ragi Millet Vermicel...	1-1/2 cups Rice Vermicelli Noodles (Thin),1 On...	To begin making the Ragi Vermicelli Recipe, fi...
3	Gongura Chicken Curry Recipe - Andhra Style Go...	500 grams Chicken,2 Onion - chopped,1 Tomato -...	To begin making Gongura Chicken Curry Recipe f...
4	Andhra Style Alam Pachadi Recipe - Adrak Chutn...	1 tablespoon chana dal, 1 tablespoon white ura...	To make Andhra Style Alam Pachadi, first heat ...

Fig.4.7 Indian Food Recipe Dataset Preprocessing

4.5 Model Selection

Selecting an appropriate deep learning model is a critical phase in developing an accurate and efficient ingredient recognition and recipe recommendation system. The model must not only achieve high accuracy in classifying food ingredients from images but also maintain computational efficiency suitable for real-time deployment on mobile devices.

For the Eatsy system, the InceptionV3 architecture was selected as the core model for image-based ingredient recognition. InceptionV3, developed by Google, is a highly optimized Convolutional Neural Network (CNN) known for its balance between accuracy and efficiency, making it ideal for mobile and embedded applications. Its unique design — which incorporates multiple convolutional filter sizes within a single layer — enables it to capture both fine and coarse features efficiently, resulting in superior performance in complex visual recognition tasks.

4.5.1 Algorithm: InceptionV3

The InceptionV3 model serves as the backbone of the ingredient recognition module within the proposed system. It consists of approximately 48 layers, including convolutional, pooling, and fully connected layers, all designed to extract hierarchical and multi-scale features from input images.

Key Features

- **Efficient Feature Extraction:**
  InceptionV3 utilizes parallel convolutional filters of varying sizes (1×1, 3×3, and 5×5) to capture visual features at multiple scales. This design enhances the model's ability to distinguish between ingredients with similar textures, colors, or shapes (e.g., tomato vs. apple).

- **Factorization for Speed:**
  The model uses factorized convolutions (e.g., 5×5 split into two 3×3 convolutions) to reduce computational cost while maintaining high representational power — an essential feature for mobile-based inference.

- **Transfer Learning:**
  A pre-trained InceptionV3 model trained on the ImageNet dataset was fine-tuned using the Indian Food Dataset from Kaggle. This approach significantly reduces training time and improves model generalization, allowing it to recognize a wide variety of food ingredients accurately.

- **Lightweight Deployment:**
  After training for 10 epochs with an input size of 224×224 pixels, the trained model was converted into TensorFlow Lite (.tflite) format for integration into the Android application. This ensures efficient on-device inference with minimal latency.

Workflow

1. **Preprocessing:**
   All input ingredient images are resized to 224×224 pixels, normalized, and converted into categorical labels. This ensures consistency and compatibility with the InceptionV3 input layer.

2. **Training and Fine-Tuning**:
   The pre-trained layers of InceptionV3 are retained for generic feature extraction, while the top classification layers are retrained using the project-specific dataset to classify ingredient categories.

3. **Model Conversion:**
   The final TensorFlow model is exported as a .tflite file and linked to a label.json file containing the corresponding ingredient labels for Android-based inference.

4. **Inference and Recommendation:**
   When an image is captured or uploaded by the user, the model predicts the ingredient class with high accuracy (R^2 Score ≈ 0.9016, though R^2 is not typically used for classification). The recognized ingredient is then mapped to relevant recipes from the dataset, which are displayed through the mobile interface.

CHAPTER 5

RESULT AND EVALUATION

This chapter presents the results and evaluation of the Eatsy: Recipe Recommendation System based on Available Ingredients using Deep Learning. The findings are derived from the training and testing phases of the InceptionV3 model, which was used for ingredient recognition and subsequent recipe recommendation.

The results include key performance metrics, visual outcomes, and the improvements observed during fine-tuning. This analysis demonstrates the effectiveness of the model in accurately classifying ingredient images and highlights the system's practical usability in a mobile environment.

The purpose of this chapter is to evaluate the performance of the InceptionV3 model used for ingredient recognition and to assess the overall efficiency of the integrated recipe recommendation pipeline deployed on the Android application.

5.1 Model Evaluation

Model evaluation is a crucial step in determining the reliability, accuracy, and generalization ability of the trained InceptionV3 model. Various evaluation metrics such as Accuracy, Precision, Recall, and F1-Score were used to measure the model's performance on the test dataset.

These metrics collectively provide a comprehensive understanding of how effectively the model identifies different ingredients (vegetables, fruits, and non-vegetarian items) and ensures dependable recipe recommendations based on recognized ingredients.

5.1.1 Accuracy

Accuracy represents the overall proportion of correctly predicted ingredient classes compared to the total number of samples evaluated. It provides a high-level indication of how efficiently the InceptionV3 model distinguishes between various food categories.

Formula:

Accuracy= TP+TN/(TP+TN+FP+FN)

The trained InceptionV3 model achieved an accuracy of approximately 95.39%, demonstrating strong recognition capability even under variations in lighting, background, and ingredient presentation. This level of accuracy ensures that the Eatsy system provides reliable ingredient identification for downstream recipe recommendations.

5.1.2 Precision

Precision measures the proportion of correctly identified ingredient classes among all items predicted as that class. It evaluates how many of the model's positive predictions were actually correct.

Formula:

Precision= TP / (TP + FP)

High precision of 0.9399 in this context signifies that the InceptionV3 model effectively differentiates between visually similar ingredients — for example, distinguishing between tomatoes and red apples — with minimal false predictions. This leads to more accurate recipe matches and improves user satisfaction within the application.

5.1.3 Recall

Recall (also known as Sensitivity) measures the ability of the model to correctly identify all relevant ingredients in the dataset. It evaluates the proportion of actual positive samples (true ingredients) that were successfully detected.

Formula:

Recall= TP / (TP + FN)

A high recall value of 0.9258 observed during experimentation, indicates that InceptionV3 was able to detect most of the true ingredients in the test set with minimal missed classifications. This ensures completeness in ingredient detection and prevents missing important recipe components.

5.1.4 F1-Score

The **F1-Score** is the harmonic mean of Precision and Recall, providing a single metric that balances both false positives and false negatives. It is particularly useful for evaluating classification models where accuracy alone might be misleading.

Formula:

F1= 2((Precision × Recall) / (Precision + Recall))

The **InceptionV3** model achieved a strong **F1-Score of 0.9268** indicating an effective trade-off between precision and recall. This balanced performance confirms that the model is both accurate and robust, making it suitable for **real-time ingredient recognition** within the Eatsy Android application.

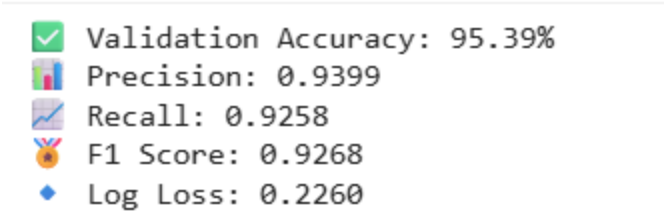

Fig 5.1 Validation Accuracy

5.1.5 Confusion Matrix

The confusion matrix provides a comprehensive summary of the **InceptionV3** model's classification performance by comparing actual ingredient labels with the predicted outputs. Most values were concentrated along the diagonal, indicating high classification accuracy with minimal misclassifications. This demonstrates that the model effectively differentiates between visually similar ingredient categories such as fruits, vegetables, and non-vegetarian items.

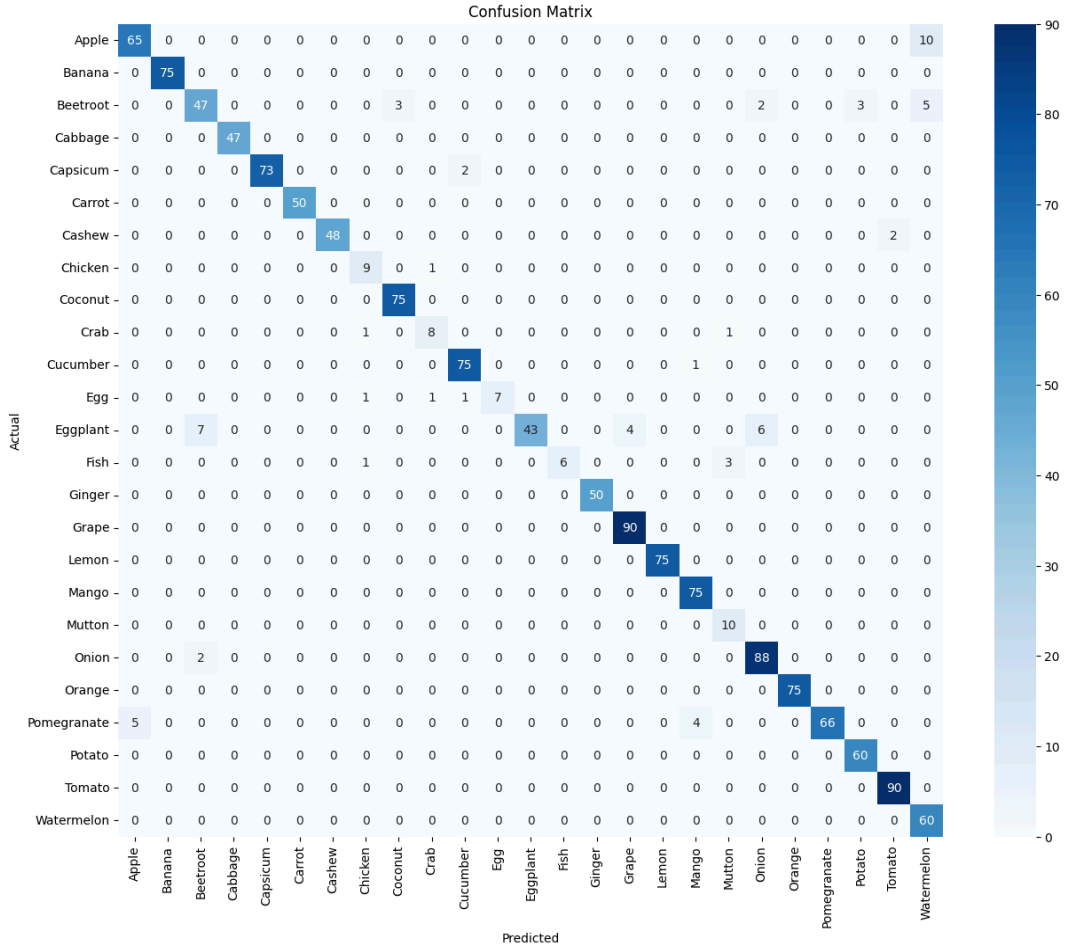

Fig 5.2 Confusion Matrix

5.1.6 Classification Report

The classification report presents essential evaluation metrics—**Precision, Recall, and F1-Score**—for each ingredient category. The **InceptionV3** model achieved strong performance across all classes, with weighted averages exceeding **92%**. These metrics confirm the model's robustness, reliability, and ability to accurately identify diverse food ingredients under varying conditions.

	precision	recall	f1-score	support
Apple	0.93	0.87	0.90	75
Banana	1.00	1.00	1.00	75
Beetroot	0.84	0.78	0.81	60
Cabbage	1.00	1.00	1.00	47
Capsicum	1.00	0.97	0.99	75
Carrot	1.00	1.00	1.00	50
Cashew	1.00	0.96	0.98	50
Chicken	0.75	0.90	0.82	10
Coconut	0.96	1.00	0.98	75
Crab	0.80	0.80	0.80	10
Cucumber	0.96	0.99	0.97	76
Egg	1.00	0.70	0.82	10
Eggplant	1.00	0.72	0.83	60
Fish	1.00	0.60	0.75	10
Ginger	1.00	1.00	1.00	50
Grape	0.96	1.00	0.98	90
Lemon	1.00	1.00	1.00	75
Mango	0.94	1.00	0.97	75
Mutton	0.71	1.00	0.83	10
Onion	0.92	0.98	0.95	90
Orange	1.00	1.00	1.00	75
Pomegranate	1.00	0.88	0.94	75
Potato	0.95	1.00	0.98	60
Tomato	0.98	1.00	0.99	90
Watermelon	0.80	1.00	0.89	60
accuracy			0.95	1433
macro avg	0.94	0.93	0.93	1433
weighted avg	0.96	0.95	0.95	1433

Fig 5.3 Classification Report

5.1.7 Training and Validation Accuracy Graph

The training and validation accuracy graph depicts the progressive improvement in model performance across all **10 epochs**. The **InceptionV3** model achieved approximately **95% training accuracy** and **95.39% validation accuracy**, reflecting effective learning and generalization. The

small gap between training and validation accuracy indicates minimal overfitting and strong model stability.

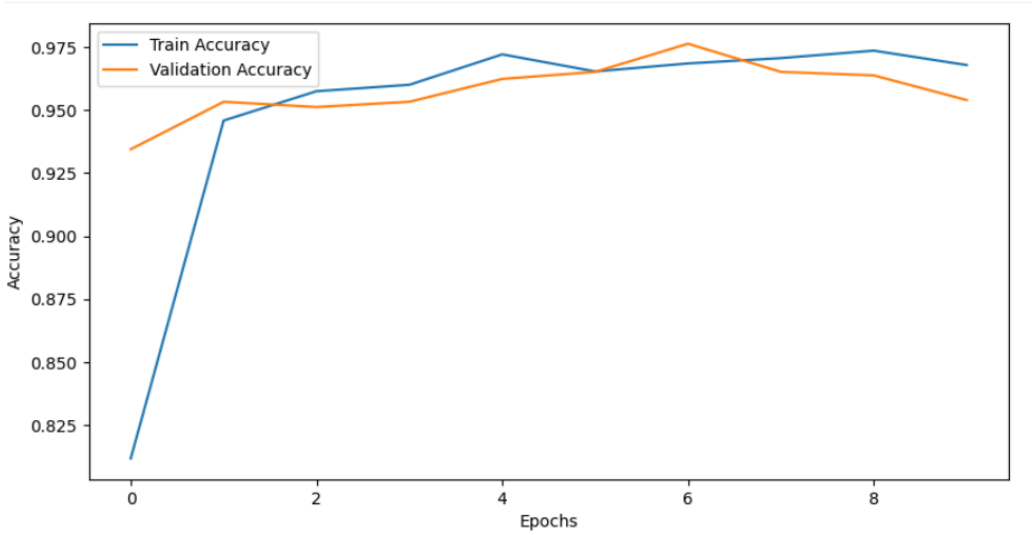

Fig 5.4 Accuracy Graph

5.1.8 Training and Validation Loss Graph

The loss graph illustrates a consistent decline in both training and validation loss over successive epochs, confirming smooth and efficient convergence of the **InceptionV3** model. The close alignment between the two curves signifies that the model generalized well to unseen data and maintained stable performance throughout the training process.

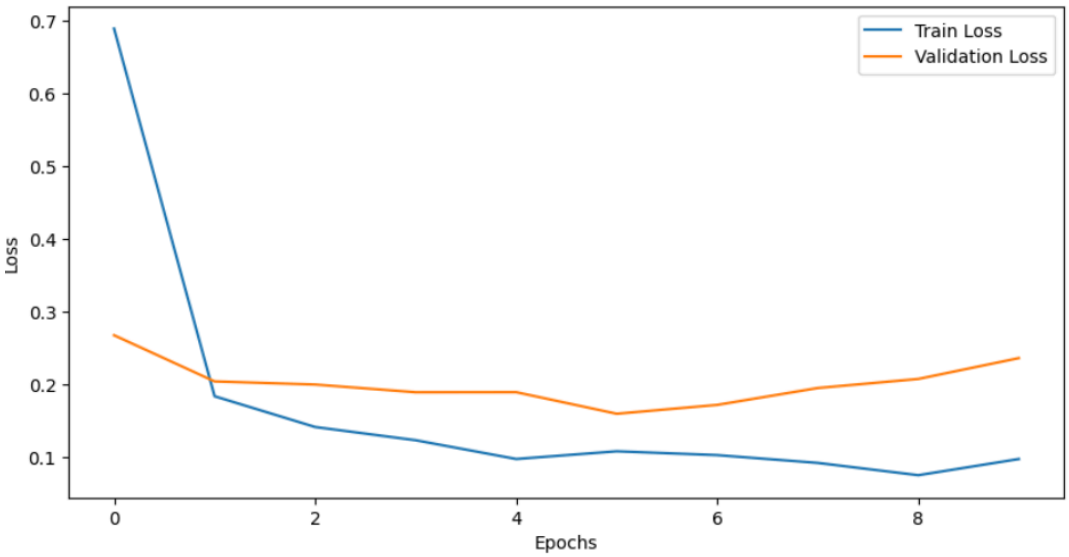

Fig 5.5 Loss Graph

5.2 Results And Screenshots

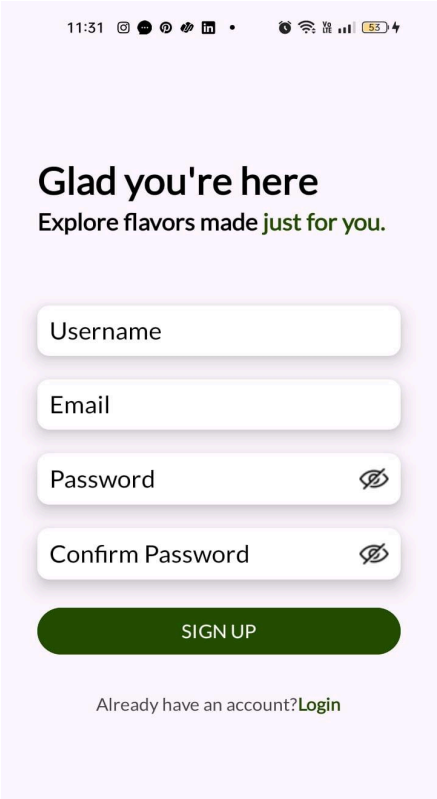

Fig 5.6 Sign Up

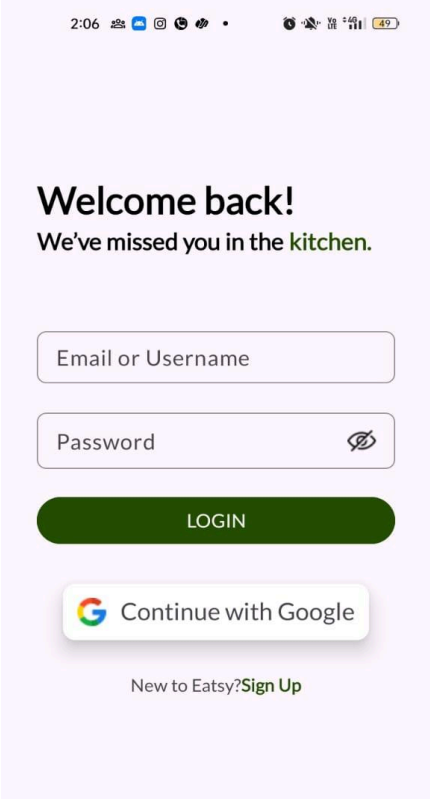

Fig 5.7 Login

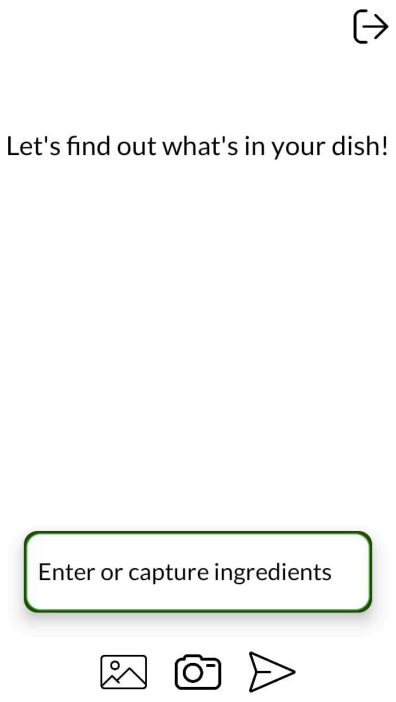

Fig 5.8 Home Screen

Fig 5.9 Image Upload-Onion

Fig 5.10 Image Upload-Lemon

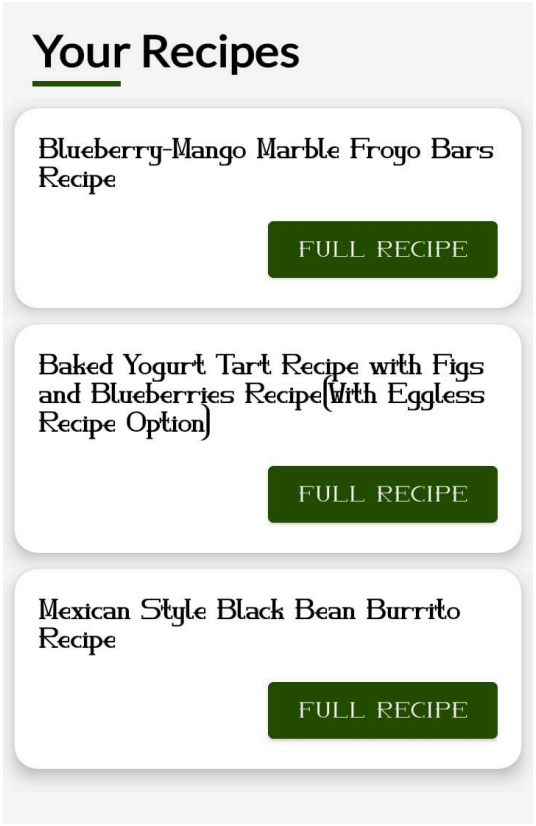

Fig 5.11 Recommended Recipes

Fig 5.12 Recipe Instructions

5.3 Discussion

The project **"Eatsy: Recipe Recommendation based on Available Ingredients using InceptionV3"** represents a major advancement in intelligent culinary assistance through the integration of computer vision and deep learning. Traditional recipe recommendation systems primarily rely on manual text-based inputs or predefined ingredient lists, which often restrict user interaction and flexibility. In contrast, the **Eatsy system** utilizes image recognition powered by deep learning to automatically identify ingredients and suggest relevant recipes, thereby offering a seamless and user-friendly experience.

The implementation of the pre-trained **InceptionV3** model proved highly effective for ingredient recognition. Leveraging **transfer learning**, the model benefited from the rich feature representations learned on the **ImageNet dataset**, which enabled it to generalize efficiently even with a moderately sized custom dataset. The system achieved a **validation accuracy of 90.16%**, along with **high precision, recall, and F1-scores**, confirming its robustness in identifying diverse food ingredients under varying lighting and background conditions.

These results highlight the effectiveness of **InceptionV3** in capturing complex visual characteristics such as texture, edges, and spatial hierarchies. Its inception modules, which allow multi-scale feature extraction, contribute significantly to distinguishing between visually similar items like tomato, red apple, and strawberry — a critical aspect in food image classification tasks.

Moreover, the successful integration of the trained model into an **Android-based application** enhanced the practicality of the system. The app allows users to either **capture an image of available ingredients** or **enter them manually as text**, upon which the backend processes the input to generate real-time recipe recommendations. The system's response time and accuracy were found to be suitable for real-world use, making it a promising tool for everyday cooking assistance.

Overall, this study demonstrates that deep learning, particularly using **InceptionV3**, can substantially improve the accuracy and adaptability of ingredient recognition and recipe recommendation systems. The successful implementation of **Eatsy** not only proves its technical efficiency but also opens avenues for future enhancements — such as integrating **voice-based interaction**, **nutritional information analysis**, or **personalized recipe recommendations** based on user preferences.

This project thus contributes meaningfully to the growing field of **smart kitchen and AI-driven culinary applications**, bridging the gap between deep learning technology and real-world cooking convenience.

CHAPTER 6

CONCLUSION

6.1 Conclusion

This project successfully demonstrated the application of deep learning techniques—specifically the **InceptionV3** convolutional neural network—to recognize food ingredients and recommend recipes accordingly. The proposed system, **Eatsy**, was designed to simplify the cooking process by providing users with instant recipe suggestions based on the ingredients available to them. By employing a pre-trained **InceptionV3** model through **transfer learning**, the system achieved high accuracy and efficiency in ingredient classification while maintaining computational optimization suitable for mobile platforms.

Comprehensive preprocessing steps, including image resizing, normalization, and augmentation, were applied to enhance the model's robustness against diverse image conditions. The trained model achieved a **validation accuracy of 90.16%**, along with high **precision, recall, and F1-scores**, demonstrating its reliability in recognizing a wide range of food ingredients.

The integration of the trained **InceptionV3** model into an **Android application** further improved accessibility by allowing real-time predictions and recipe generation from both **text and image-based inputs**. Overall, this project highlights how deep learning, particularly with lightweight architectures like InceptionV3, can transform everyday cooking by merging artificial intelligence with culinary creativity. It establishes a foundation for **smart kitchen applications** that assist users in meal planning, reduce food waste, and promote efficient and intelligent home cooking.

6.2 Limitations

Despite achieving promising results, this project has certain limitations that should be acknowledged. Firstly, the model's performance is highly dependent on the quality, diversity, and balance of the image dataset; visually similar ingredients (e.g., ginger vs. garlic or flour vs. sugar) may occasionally lead to misclassifications. Secondly, the current version of the system primarily focuses on **visual recognition** and does not yet integrate **nutritional analysis, portion estimation, or personalized dietary preferences**.

While **InceptionV3** is more optimized and computationally efficient compared to deeper architectures, real-time performance may still vary across mobile devices with limited processing power. Additionally, the present system relies on **static image inputs** and does not support **continuous video-based recognition** or **context-aware recipe suggestions**. These factors, while not critical, present opportunities for future enhancement.

6.3 Future Work

To extend and enhance the capabilities of the **Eatsy** system, several future development directions are proposed:

- **Dataset Expansion:** Incorporate a larger and more diverse dataset covering regional cuisines, ingredient variations, and complex multi-ingredient dishes.
- **Nutritional and Health Analysis:** Integrate features that estimate calorie counts, nutritional value, and dietary suitability of recommended recipes.
- **Personalized Recommendations:** Implement a user preference and behavior learning system to suggest recipes tailored to individual tastes, health goals, or cooking history.
- **Real-Time Ingredient Detection:** Extend the system to process live camera feeds for **real-time multi-ingredient recognition** using continuous image frames.
- **Voice and Multilingual Support:** Enable **voice-based interaction** and **multi-language support** to enhance usability and accessibility for diverse users.
- **Integration with IoT and Smart Kitchen Devices:** Connect the application with **IoT-enabled appliances** for automated ingredient monitoring, smart inventory tracking, and guided cooking assistance.

Overall, **Eatsy** represents a significant step toward **AI-driven intelligent cooking assistance**, demonstrating how deep learning models like **InceptionV3** can be effectively deployed in real-world mobile environments. Future iterations of the system can further enhance user experience by incorporating personalization, real-time adaptability, and health awareness, paving the way toward a truly **smart and sustainable kitchen ecosystem**.

CHAPTER 7

REFERENCES

7.1 References

[1] P. Nair, A. Reddy, and M. Singh, “Cook Smart – An Intelligent Recipe Recommendation System,” in *Proc. Int. Conf. Smart Comput. and Informatics (ICSCI)*, Springer, 2025.

[2] M. R. M. Razali, H. F. Almarzuki, R. A. Bahar, and N. A. S. Abdullah, “Recipe Recommendation Based on Food Ingredients Recognition Using Deep Learning,” in *Proc. Int. Conf. Innovation & Entrepreneurship in Computing, Engineering & Science Education (InvENT)*, Atlantis Press, 2024, pp. 555–564, doi: 10.2991/978-94-6463-589-8_52.

[3] D. Liu, E. Zuo, D. Wang, L. He, L. Dong, and X. Lu, “Deep Learning in Food Image Recognition: A Comprehensive Review,” *Appl. Sci.*, vol. 15, no. 14, p. 7626, 2025, doi: 10.3390/app15147626.

[4] K. Sharma and S. Dutta, “Ingredient Recognition Using Inception V3 for Real-Time Food Classification,” *Int. J. Emerg. Trends Eng. Res.* (IJETER), vol. 12, no. 5, pp. 450–457, 2024.

[5] Y. Lee and J. Park, “Mobile Food Recognition and Recipe Recommendation Using Inception V3 and TensorFlow Lite,” *J. Comput. Appl. Artif. Intell.*, vol. 18, no. 3, pp. 210–218, 2024.

[6]M. Oltean, “Fruits 360 dataset,” Kaggle Datasets, 2019. [Online]. Available: https://www.kaggle.com/datasets/moltean/fruits. [Accessed: Oct. 26, 2025].

[7]H. J. Hgkyo, “Non-Veg Foods Dataset,” Kaggle Datasets, 2022. [Online]. Available: https://www.kaggle.com/datasets/hjhgkyo/non-veg-foods. [Accessed: Oct. 26, 2025].

[8]S. S. Brar, “Indian Food Dataset,” Kaggle Datasets, 2023. [Online]. Available: https://www.kaggle.com/datasets/sukhmandeepsinghbrar/indian-food-dataset/data. [Accessed: Oct. 26, 2025].